

KnowAct: Evaluating Functional Theory of Mind through Active Knowledge-State Diagnosis

Anonymous authors
Paper under double-blind review

Abstract

Large language model agents increasingly collaborate with users in learning, research, and decision-making tasks, where effective assistance depends on understanding what the user knows, misunderstands, or remains uncertain about. Existing Theory-of-Mind (ToM) benchmarks primarily evaluate mental-state inference from fixed narratives or conversations, while recent interactive benchmarks emphasize social goal completion, persuasion, or personalization. These settings do not isolate a complementary capability: whether an agent can actively acquire evidence about a user’s structured knowledge state and use its evolving user model to decide what to ask next. We introduce KnowAct, a benchmark framework for evaluating functional ToM in knowledge-grounded interaction. Each bounded episode combines a source-grounded domain knowledge graph, a hidden user knowledge map, simulator-only evidence, and a fixed interaction budget. A tested agent selects diagnostic questions, maintains a full-graph working knowledge map, and submits a final reconstruction of the hidden user state. To make evaluation reproducible and auditable, KnowAct separates visible and hidden contexts, constrains the user simulator through a de-identified answer blueprint, and scores profile reconstruction with deterministic node-level metrics rather than an LLM judge. The initial benchmark domain is statistical learning, grounded in an authoritative textbook. This paper presents the benchmark formulation, construction pipeline, simulator design, agent protocol, and evaluation methodology; empirical results are left for subsequent work.

1 Introduction

Language-model agents are moving from single-turn question answering toward long-horizon collaboration. In such settings, the external task state is only part of the problem. An effective agent must also reason about the user: which concepts are already understood, which distinctions are fragile, which misconceptions are present, and what interaction would be most informative or helpful next. This requirement is closely related to Theory of Mind (ToM), broadly understood as reasoning about the mental states of others.

Most machine-ToM evaluations, however, are endpoint prediction tasks. A model reads a narrative or dialogue and answers a question about a character’s belief, intention, emotion, or likely behavior (????). These benchmarks have substantially improved the rigor and breadth of ToM evaluation, but they usually hold the evidence-gathering process fixed: the model cannot choose what to ask, decide which uncertainty to resolve, or update an explicit user model over multiple turns. Moreover, recent results suggest a gap between explicitly inferring a mental state and applying that inference to downstream behavior or judgment (?).

Interactive social-agent environments and applied-ToM methods begin to address this gap. They evaluate goal-oriented dialogue, strategic behavior, persuasion, and user alignment (????). Yet these settings typically optimize holistic task outcomes or social goals rather than the accurate recovery of a controlled latent user state. Consequently, a successful

interaction may not reveal whether the agent formed an accurate user model, used a shortcut, or merely produced a generally effective response.

We study a narrower but measurable question:

Can an agent actively infer a user’s hidden knowledge state through limited natural-language interaction, and use that evolving estimate to choose subsequent diagnostic actions?

We call this capability functional ToM for knowledge-grounded interaction. The term functional emphasizes that user modeling is evaluated as part of a closed loop: the agent must infer, update, and act, rather than only verbalize a mental-state description. The term knowledge-grounded restricts the hidden state to a reviewed domain representation with diagnosable concepts, making the target explicit enough for controlled evaluation.

To operationalize this question, we introduce KnowAct. Each episode binds four artifacts: (1) an authored domain knowledge graph, (2) a hidden user-specific knowledge map over that graph, (3) grounded evidence used only by the simulator, and (4) an immutable episode configuration including a turn budget and scoring profile. The tested agent sees the public graph and visible dialogue, but not the hidden map or simulator evidence. At each turn it asks one diagnostic question, observes the simulated user’s answer, updates a full-graph working map, and selects the next question. At the end of the episode, it submits one mastery prediction for every graph node. This design converts functional user modeling into a partially observed sequential diagnosis problem with a deterministic endpoint target.

Our contributions are:

- We formulate active knowledge-state diagnosis as a controlled evaluation setting for functional ToM, separating mental-state reconstruction from downstream social or task success.
- We introduce a semi-synthetic benchmark construction pipeline that combines source-grounded knowledge graphs, reviewed hidden user maps, evidence-backed simulation, and strict visibility boundaries.
- We define an agent protocol with an explicit full-graph working map and bounded semantic actions, together with deterministic, evidence-auditable reconstruction metrics that avoid an LLM judge in the primary score.

KnowAct does not claim to establish human-like ToM in language models. It operationalizes one practically important behavior: whether an agent can build and use a structured model of a user’s knowledge under controlled interaction.

2 Related Work

2.1 Machine Theory-of-Mind Evaluation

Early and contemporary machine-ToM benchmarks commonly frame mental-state reasoning as question answering over stories or conversations. ToMi revisited false-belief evaluation and showed that dataset regularities can inflate apparent ToM performance (?). BigToM introduced causally structured social scenarios and diverse questions about beliefs, desires, and actions (?). FANToM stresses information-asymmetric multi-party conversations and evaluates consistency across multiple question types (?). Hi-ToM extends evaluation to higher-order belief reasoning (?), while OpenToM broadens narratives, personality information, and psychological mental-state questions (?). ToMBench systematizes a wide range of social-cognitive abilities with controlled multiple-choice evaluation (?).

These benchmarks are essential for evaluating whether models can infer latent mental states from supplied evidence. However, they generally do not evaluate the acquisition policy itself: the model receives a fixed context instead of choosing which evidence to request. SimpleToM further exposes an inference–application gap, showing that models that answer explicit mental-state questions correctly may still fail to predict or judge behavior using the

inferred state (?). KnowAct builds on this observation but shifts the application target from behavior prediction to sequential diagnostic action. The agent’s user model matters because it determines which question is asked next and which parts of the hidden knowledge map can be recovered within a turn budget.

2.2 Interactive Social Agents and Applied ToM

SOTOPIA evaluates language agents in open-ended social scenarios involving cooperation, competition, negotiation, and relationship management (?). AgentSense expands scenario diversity and evaluates social goal completion and implicit reasoning in interactive settings (?). Applied-ToM methods explicitly insert mental-state reasoning into an agent loop. ToMAgent generates mental-state hypotheses between dialogue turns and learns representations useful for long-horizon social goals (?). ToMELP evaluates an infer-apply loop in route-controlled persuasion (?). ToM-SWE models user goals, preferences, and constraints to improve software-engineering agents under ambiguous and stateful instructions (?).

This line of work establishes that explicit user modeling can improve interaction. Its evaluation target, however, is usually an external outcome such as social goal completion, persuasive effectiveness, or software task success. Such outcomes intentionally combine many capabilities, including planning, generation quality, domain competence, and strategic communication. KnowAct instead isolates the user-modeling loop: the hidden state has an explicit structure, the agent actively probes it, and the final model can be compared node-by-node against ground truth. The two paradigms are complementary. Applied-agent benchmarks ask whether ToM improves task behavior; KnowAct asks whether the agent can construct the user model that such behavior would depend on.

2.3 Personalization, User Profiling, and Knowledge Tracing

Personalization benchmarks evaluate whether language models use user histories or profiles to produce aligned outputs. LaMP provides diverse personalized classification and generation tasks grounded in user-specific records (?). PERSONAMEM evaluates dynamic profiling over long multi-session histories and whether models apply the inferred profile to later responses (?). These benchmarks focus primarily on preferences, traits, and evolving personal context observed passively from prior interactions. In contrast, KnowAct gives the tested agent control over evidence acquisition and focuses on a domain-specific epistemic state.

Educational modeling offers a second close connection. Bayesian Knowledge Tracing models latent learner mastery from sequences of item responses (?); neural approaches such as Deep Knowledge Tracing and Attentive Knowledge Tracing improve predictive flexibility over long response histories (??). LongTutor evaluates long-term evidence acquisition, knowledge diagnosis, and adaptive teaching from expert-annotated learning logs (?). These methods infer learner state from an existing sequence or curriculum. KnowAct instead treats question selection as part of the tested capability: agents ask open-ended natural-language questions, gather qualitative evidence, and reconstruct a full graph state under a shared episode protocol. Thus, KnowAct is closer to active diagnosis than conventional next-response prediction.

2.4 LLM-Based User Simulation

User simulators make interactive evaluation scalable and reproducible, but they can introduce inconsistency, leakage, and simulator-specific artifacts. DuetSim uses separate generation and verification models to improve task-oriented user simulation (?). SimulatorArena directly studies whether LLM simulators are reliable proxies for humans and shows that profile conditioning can improve alignment, while also underscoring the need for explicit simulator validation (?). More broadly, simulator design research emphasizes that a useful simulator must model task-relevant user behavior rather than merely generate fluent turns (?).

Table 1: Positioning of KnowAct relative to adjacent evaluation paradigms. “Active probes” means the tested model chooses what evidence to request.

Paradigm	Interactive	Explicit latent user state	Active probes	Direct state score
Narrative / conversational ToM	Limited	Partial	No	Usually QA
Social-agent arenas	Yes	Usually no	Yes	No
Personalization from histories	Usually no	Profile / preference	No	Sometimes
Knowledge tracing	Sequential	Mastery	Usually no	Prediction-based
KnowAct	Yes	Knowledge map	Yes	Yes

KnowAct adopts an LLM-based simulator but narrows and structures its authority. The simulator does not freely invent a user. It is conditioned on a reviewed hidden map and grounded evidence, and hidden information is transformed into a de-identified answer blueprint before natural-language generation. Primary scoring compares the tested agent’s final structured map against the hidden map and does not rely on a simulator or evaluator model to judge success. Human validation of simulator fidelity remains necessary and is treated as an explicit limitation and planned evaluation component.

3 The KnowAct Benchmark

KnowAct evaluates whether an agent can infer, maintain, and use a structured model of a user’s knowledge. This section defines the target state, benchmark artifacts, interaction boundary, agent protocol, and scoring rule.

3.1 Active Knowledge-State Diagnosis

We operationalize functional ToM as three observable abilities: infer a user state from interaction evidence, revise that state over time, and use the current estimate to select the next action.

The benchmark does not assume a particular cognitive architecture. It tests these abilities through bounded, auditable behavior in a controlled environment.

Domain knowledge graph. A benchmark domain is represented by an authored knowledge graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad (1)$$

where each node $v_i \in \mathcal{V}$ is a diagnosable knowledge unit and each edge $e_{ij} \in \mathcal{E}$ is a reviewed domain relation.

Nodes contain definitions, source locators, diagnostic goals, mastery rubrics, diagnostic signals, and simulator guidance. Edges organize the domain but do not encode user state.

Hidden user knowledge map. For a graph with $N = |\mathcal{V}|$ nodes, the hidden map is

$$\mathcal{M}^* = \{s_i^*\}_{i=1}^N. \quad (2)$$

Each state includes mastery $m_i^* \in \{0, 1, \dots, 5\}$, evidence references, misconceptions, and unknowns. Levels are node-specific rubric positions, not a universal psychometric scale.

Optional profile context can control conversational style. It is not part of the primary mastery target and cannot override the reviewed node states.

Bounded diagnostic episode. An episode is specified by

$$\mathcal{E}p = (\mathcal{G}, \mathcal{M}^*, \mathcal{Z}, T, \rho, \sigma), \quad (3)$$

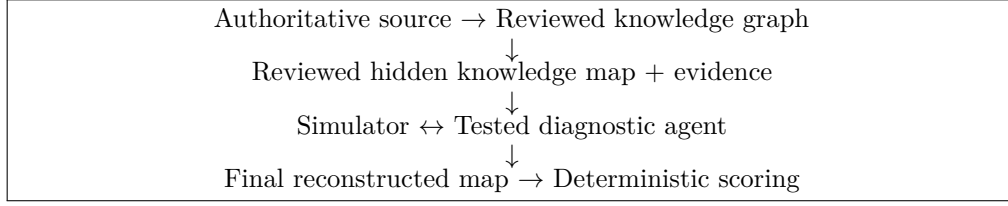


Figure 1: The KnowAct pipeline. Authoring produces reviewed hidden artifacts; the tested agent interacts only through the visible episode interface.

where $\mathcal{Z}$ is simulator-only evidence, T is the turn budget, ρ is the interaction rule, and σ is the scoring profile.

At turn t , the agent observes the public graph, visible history $\mathcal{H}_{t-1}$, and its working map $\widehat{\mathcal{M}}_{t-1}$. It chooses one diagnostic question

$$q_t \sim \pi_\theta(\mathcal{G}, \mathcal{H}_{t-1}, \widehat{\mathcal{M}}_{t-1}), \quad (4)$$

receives answer a_t , and updates its estimate

$$\widehat{\mathcal{M}}_t = U_\theta(\widehat{\mathcal{M}}_{t-1}, q_t, a_t). \quad (5)$$

After at most T turns, the agent submits a full-graph reconstruction $\widehat{\mathcal{M}}_T$. The hidden map, evidence, simulator traces, and scoring internals remain inaccessible throughout the episode.

3.2 Benchmark Construction

Source-grounded graph authoring. The initial domain is statistical learning, grounded in *An Introduction to Statistical Learning with Applications in Python* (?).

Parsed source segments support node extraction, cross-segment reconciliation, diagnostic enrichment, and edge proposal. Models propose artifacts, while human review determines which versions are promoted.

Source locators preserve provenance and enable audits of definitions and rubrics. Promoted graph snapshots are immutable, preventing later authoring changes from silently altering an episode.

Knowledge-map authoring. A hidden map assigns a reviewed state to every graph node. Evidence records specify what the user can explain, where an explanation fails, and which misconceptions or uncertainties may surface.

Authoring separates three artifacts:

- the static node-level state used as scoring ground truth;
- simulator-only evidence that supports natural answers;
- optional profile context that controls style without changing knowledge.

Candidate maps undergo human checks for graph coverage, internal consistency, plausibility, and evaluability before they become eligible for episodes.

Episode manifests. An immutable manifest binds the graph version, hidden map, turn budget, interaction rule, scoring profile, and pinned provider configuration.

This binding makes repeated runs comparable and prevents accidental changes in benchmark artifacts or runtime settings. Appendix B provides the detailed authoring workflow.

3.3 User Simulation and Visibility Boundary

The simulator must produce natural answers while remaining faithful to the hidden map. KnowAct therefore treats simulation as a staged information boundary rather than an unconstrained role-playing prompt.

Question grounding. A grounder maps the current question to a directly targeted node set $S_t \subseteq \mathcal{V}$. It also detects ungrounded requests, hidden-label requests, and multiple independent questions.

Grounding sees the public graph and limited visible dialogue, but no hidden state. A probe may cover multiple nodes only when they form one integrated diagnostic question, such as a comparison.

Hidden-context selection and answer policy. After grounding, the context builder retrieves state and evidence only for nodes in S_t . Connected neighbors are not included merely because a graph edge exists.

An answer policy converts the selected context into a de-identified blueprint containing expressible claims, uncertainty boundaries, misconceptions, evidence-supported cues, and prohibited overstatements.

The blueprint excludes mastery labels, node and evidence identifiers, map identifiers, user identifiers, and scoring fields. It is the sole content interface to natural-language generation.

Generation and leakage control. The generator receives the blueprint, visible dialogue required for continuity, and an optional style hint. It never receives the raw hidden map.

Generation preserves partial knowledge and misconceptions rather than reducing all states to know/do-not-know answers. Bounded retries and safe fallbacks handle malformed outputs.

The runtime separates tested-agent, simulator, and evaluation contexts. Hidden policy traces remain available to benchmark authors for audit but never enter the visible transcript.

3.4 Active Diagnosis Agent Protocol

Full-graph working map. The tested agent maintains an assessment for every node:

$$\hat{s}_{i,t} = (\hat{m}_{i,t}, c_{i,t}, n_{i,t}, R_{i,t}), \quad (6)$$

where $\hat{m}_{i,t} \in \{\text{unknown}, 0, \dots, 5\}$ is assessed mastery, $c_{i,t}$ is confidence, $n_{i,t}$ is an optional note, and $R_{i,t}$ contains supporting visible turns.

All nodes begin as unknown. A non-unknown assessment should cite visible evidence; unsupported estimates may be submitted but are reported separately.

The full map exposes uncertainty and gives the policy a domain-wide basis for comparing mastery, confidence, graph position, and evidence coverage.

Semantic action interface. The runtime exposes only benchmark-relevant operations:

- read the authored episode graph or current working map;
- update one node assessment with visible support;
- ask one diagnostic question;
- finalize the reconstructed map.

These actions make state updates and probe choices observable. They also prevent policies from bypassing the intended loop through generic access to hidden artifacts.

Diagnostic loop. A tested agent repeatedly:

1. interprets the latest answer as evidence about visible concepts;

2. revises mastery, confidence, notes, and supporting turns;
3. selects a probe expected to reduce decision-relevant uncertainty;
4. asks one question or finalizes when the budget is exhausted.

KnowAct does not prescribe how information value is computed. Agents may use direct prompting, graph-aware coverage, Bayesian updates, planning, or learned policies under the same interface.

3.5 Evaluation Metrics

Primary reconstruction metric. Let the final predicted mastery for node i be $\hat{m}_i$. For submitted levels 0 through 5, the node loss is squared ordinal distance:

$$d(\hat{m}_i, m_i^*) = (\hat{m}_i - m_i^*)^2. \quad (7)$$

An unknown prediction receives the fixed maximum penalty of 36. The episode score is

$$D_{\text{mastery}} = \frac{1}{N} \sum_{i=1}^N d(\hat{m}_i, m_i^*), \quad (8)$$

where lower is better. The score is deterministic and requires no evaluator model.

Ordinal distance rewards near misses and penalizes severe errors more strongly than exact match. Scores must still be interpreted against node-specific rubrics rather than as universal psychometric distances.

Supporting metrics. Secondary metrics characterize both endpoint quality and diagnostic behavior:

- exact mastery match, signed error, and missing-prediction rate;
- unsupported inference rate based on absent visible support;
- graph coverage and reconstruction quality by turn count;
- redundant or ungrounded question rate;
- information gain per turn for agents that expose calibrated beliefs.

Misconceptions and free-text unknowns support qualitative analysis but are excluded from the initial automatic score, avoiding an additional subjective judge.

4 Experimental Evaluation

The empirical evaluation is organized around the benchmark’s central claim: adaptive evidence acquisition should improve hidden-map reconstruction under a fixed interaction budget.

4.1 Research Questions

We ask four questions:

1. Do adaptive agents outperform fixed and random diagnostic policies under equal turn budgets?
2. Do explicit working maps and graph relations improve reconstruction and question efficiency?
3. Are agent rankings robust to simulator model, answer policy, grounding, and profile style?
4. Do simulator answers and agent rankings agree with matched human-user episodes?

These questions separate endpoint reconstruction, state-conditioned action, simulator dependence, and transfer to human interaction.

4.2 Benchmark Instances and Experimental Setup

The initial study uses reviewed statistical-learning graphs grounded in the ISLP source, reviewed hidden maps, and immutable episode manifests.

Each reported run must identify the graph and map versions, turn budget, interaction rule, scoring profile, simulator configuration, tested-agent model, and provider settings.

All compared agents receive identical public graphs and episode constraints. Hidden maps, simulator evidence, policy traces, and scoring internals remain outside the tested-agent context.

4.3 Baselines

The initial evaluation compares three baseline families:

- Fixed-question agent: follows a predefined diagnostic order and does not adapt to answers.
- Random-question agent: samples valid diagnostic targets under the episode constraints.
- Simple LLM agent: chooses questions and updates its map directly through the semantic interface.

These controls distinguish the value of interaction from the value of adaptive user modeling. Structured agents can add uncertainty estimates, graph-aware planning, or explicit information-gain objectives.

4.4 Main Reconstruction Results

The main comparison reports mastery distance, exact match, missing predictions, and unsupported inference for every agent under matched budgets.

Results should include confidence intervals across episode instances and repeated stochastic runs. Per-node errors should accompany aggregate scores to expose systematic over- or under-estimation.

4.5 Efficiency, Ablations, and Robustness

Turn-efficiency curves measure reconstruction quality after each interaction. They test whether adaptive policies gain information earlier rather than only matching non-adaptive agents at the final turn.

Ablations remove the explicit working map, graph relations, or evidence requirements. Robustness checks vary simulator models, grounding policies, answer policies, and profile styles.

4.6 Simulator Validation and Error Analysis

Simulator validation compares generated answers with matched human responses on state fidelity, diagnostic usefulness, naturalness, and leakage.

Agent error analysis groups failures into poor grounding, missed evidence, unsupported graph propagation, stale working-map state, redundant probes, and premature finalization.

5 Discussion and Conclusion

5.1 What KnowAct Measures

Low reconstruction error is not evidence of functional ToM when it can be obtained through leakage or a fixed prior. KnowAct therefore pairs endpoint accuracy with an observable diagnostic process.

The visibility boundary limits available evidence, questions consume a fixed budget, map updates cite visible turns, and non-adaptive agents provide controls. Together, these constraints distinguish:

1. state inference: interpreting what an answer implies;
2. state maintenance: integrating evidence over time;
3. state-conditioned action: using the current map to select a probe.

The interaction is a form of active information acquisition over a structured latent state. Unlike pool-based active learning, actions are open-ended questions interpreted by a conversational simulator.

Unlike conventional adaptive testing, the probe set need not be fixed or calibrated in advance. This flexibility motivates the benchmark while making grounding, evidence audit, and simulator validation essential.

The benchmark is diagnostic rather than task-complete. It does not yet test whether a reconstructed map improves teaching, recommendation, or collaboration.

5.2 Limitations and Threats to Validity

Simulator fidelity. A simulator can be consistent with a reviewed map without matching how people express partial knowledge. Human studies must test response similarity, diagnostic value, and transfer of agent rankings.

Construction circularity. Related model families may author artifacts, simulate users, and serve as tested agents. Correlated biases could make episodes easier or distort rankings.

Source grounding, human review, model-family diversity, rule-based fixtures, and matched human validation reduce this risk but cannot eliminate it.

Simplified knowledge state. A static L0–L5 state cannot capture context dependence, procedural fluency, forgetting, confidence, or learning during interaction.

The initial benchmark freezes the hidden state to make reconstruction identifiable. Dynamic learning and state transition are important extensions.

Initial domain scope. The first release focuses on statistical learning. Evaluation across domains is required to test transfer across graph topologies, source styles, and concept distributions.

Operational claims. Success demonstrates behavior compatible with structured user modeling under benchmark constraints. It does not establish human-like beliefs, consciousness, or a psychologically faithful ToM mechanism.

5.3 Conclusion and Outlook

We introduced KnowAct, a benchmark for evaluating functional ToM through active knowledge-state diagnosis.

A tested agent sees a reviewed domain graph but not the user-specific map. Under a fixed budget, it must ask diagnostic questions, maintain an evidence-backed working map, and reconstruct the hidden state.

A visibility-separated simulator and deterministic scoring protocol make the process auditable. The design isolates active user-state acquisition from broader social or task success.

The next step is empirical validation across adaptive and non-adaptive agents, simulator designs, matched human episodes, and additional knowledge domains.

Ethics Statement

User modeling can improve tutoring, accessibility, and collaborative assistance. It can also enable manipulation, surveillance, or unjustified inference about individuals.

KnowAct uses synthetic or explicitly reviewed benchmark profiles. It should not be interpreted as permission to infer sensitive attributes from real users without informed consent.

Derived systems should minimize retained personal data, expose uncertainty, and let users inspect or correct inferred states. Knowledge estimates should not determine high-stakes outcomes without human oversight.

Benchmark releases should document source provenance and licenses. Profile contexts, evidence records, and released traces must not contain identifiable personal information.

A Implementation-Oriented Artifact Schemas

The abbreviated schemas below summarize the benchmark contracts. Field names may evolve, but the separation among public graph, hidden map, visible observations, and final reconstruction should remain stable.

A.1 Knowledge Node

```
{
  "id": "active_learning",
  "name": "Active Learning",
  "type": "concept",
  "definition": "...",
  "source_locators": [...],
  "diagnostic_goal": "...",
  "levels": {"L0": "...", ..., "L5": "..."},
  "diagnostic_signals": [...],
  "simulator_behavior": "...
}
```

A.2 Hidden User State

```
{
  "node_id": "active_learning",
  "mastery_level": "L2",
  "evidence_refs": ["ev_104"],
  "misconceptions": [...],
  "unknowns": [...]
}
```

A.3 Agent Working Assessment

```
{
  "node_id": "active_learning",
  "assessed_mastery_level": "L2",
  "diagnostic_confidence": "medium",
  "assessment_note": "Understands the motivation but not
    query-strategy trade-offs.",
  "supporting_turn_ids": ["turn_03"]
}
```

B Benchmark Construction Details

B.1 Graph-Authoring Workflow

The graph-authoring workflow has four reviewed stages:

1. Node extraction. Segment-level calls propose diagnosable units with source locators and grounding notes.
2. Reconciliation. Candidate nodes are merged, split, and deduplicated while preserving provenance.
3. Diagnostic enrichment. Each node receives a definition, diagnostic goal, rubric, diagnostic signals, and simulator guidance.
4. Edge proposal and review. Relations are proposed from the complete node set and manually reviewed before promotion.

The relation vocabulary includes `part_of`, `prerequisite_for`, `supports`, and `contrasts_with`. Candidate confidence does not replace human review.

B.2 Knowledge-Map Review

Reviewers verify that each candidate map covers every node and that each state is supported by behaviorally realizable evidence.

They also check consistency among mastery, misconceptions, unknowns, profile context, and graph structure. Promotion creates an immutable reviewed snapshot.

B.3 Manifest-Bound Runtime Contexts

The runtime maintains three contexts:

- Tested-agent context: public graph, interaction rules, visible transcript, and working map.
- Simulator context: grounded hidden states, evidence, limited visible dialogue, and optional style context.
- Evaluation context: hidden map, final reconstruction, and deterministic scoring configuration.

An episode manifest fixes the artifact versions and execution settings shared by these contexts. Debug traces and hidden policy outputs never enter the tested-agent context.

C Simulator and Agent Interface Details

C.1 Simulator Stage Contracts

The question grounder receives the public graph, the current question, and limited visible history. It returns grounded node identifiers or a structured rejection reason.

The answer policy receives only the grounded hidden context. It returns a de-identified blueprint that constrains claims, uncertainty, misconceptions, cues, and prohibited overstatements.

The generator receives the blueprint and surface-style context. It produces the user-visible answer without access to raw mastery labels, map identifiers, evidence identifiers, or scoring fields.

C.2 Tested-Agent Actions

The tested agent can read the public graph and its working map, update a node with visible support, ask one grounded question, or finalize.

Each mutation and question is recorded in the episode trace. Exact prompt templates, structured schemas, and provider-specific settings should accompany the released benchmark artifact.